

Copernicus 功能清单

作者: afu

本文档基于系统实际代码梳理，覆盖从文件上传到最终输出的完整功能链路。

一、系统总览

Copernicus 是一套音视频合规审核工作台，核心能力包括：

- 音频/视频文件上传与去重（普通上传 + 断点续传）
- ASR 语音识别（双模式）
- 四阶段文本纠正
- 动态纪要模板驱动的 Map-Reduce 内容评估
- 视频关键帧提取 + OCR 扫描 + 人脸检测
- 多源合规审核（语音 / OCR / 视觉）
- 违规人工复核与持久化
- 多说话人 TTS 音频重塑（ChatTTS）

技术栈：FastAPI + React 19 + TypeScript + Zustand + FunASR + Ollama + ChatTTS + RapidOCR + YOLO

二、功能流程总览

文件上传（普通 / 分片断点续传）→ SHA-256 去重判定

↓

视频预处理（提取音频 + 保存视频，仅视频任务）

↓

视觉扫描（仅视频且 visual_scan=true）

关键帧提取 → OCR 扫描 → 人脸检测

↓

音频预处理（格式转换 + 归一化）

↓

ASR 语音识别（Paraformer / SenseVoice）

↓

说话人平滑 + 预合并

↓

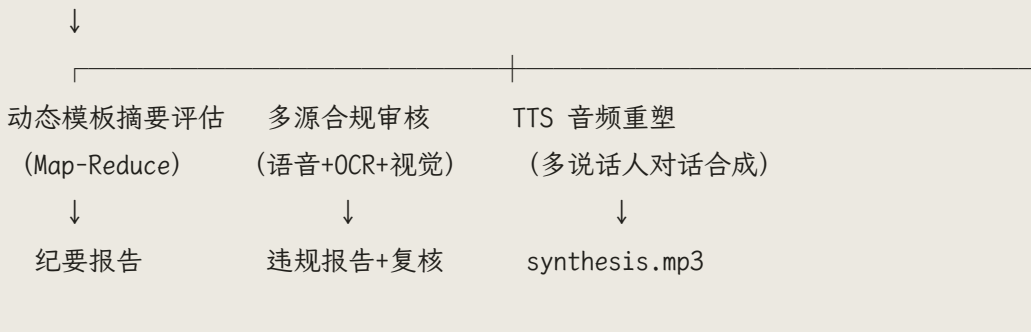
四阶段文本纠正

↓

转写结果构建

↓

文件持久化 (JSON + 媒体文件)



三、功能清单

3.1 文件上传与去重

功能	说明
拖拽/点击上传	UploadPage 支持拖拽区域和点击选择, 接受 audio/* 和 video/*
SHA-256 去重	上传前用 hash-wasm 分块计算文件哈希 (8 MB/块, 大文件不整体读入内存), 命中已有任务直接复用结果
小文件直传	小于 20 MB 的文件通过 POST /tasks/standard_minutes 一次性上传
大文件分片断点续传	大于 20 MB 的文件通过 GET /uploads/{hash} 预检 + PATCH /uploads/{hash} 分块上传 (5 MB/块), 支持中断后从断点继续
视频自动识别	根据文件扩展名自动进入视频处理流程, 弹窗询问是否启用视觉扫描
纪要模板选择	上传前可选择纪要模板 (加载自 GET /templates), 随文件一起提交给后端
异步任务创建	上传后立即返回 task_id, 后端异步执行 Pipeline, 前端轮询进度

对应端点: POST /api/v1/tasks/standard_minutes (小文件)、GET + PATCH /api/v1/uploads/{hash} (大文件分片)

3.2 Pipeline 流水线 (9 个 Stage)

Pipeline 采用插件化架构, 每个 Stage 实现统一的 Protocol 接口, 由 Orchestrator 顺序编排执行。每个 Stage 通过 should_run 方法决定是否跳过。

Stage 1: 视频预处理 (VideoPreprocessStage)

功能	说明
视频音频提取	ffmpeg 将视频中音频流提取为 16kHz 单声道 WAV
音频增强滤镜	应用 highpass （去低频噪声）+ afftdn （FFT 降噪）+ dynaudnorm （动态归一化）
媒体类型标记	设置 media_type = “video”，后续 Stage 和前端据此调整行为

跳过条件: 非视频文件时自动跳过

Stage 2: 关键帧提取 (KeyframeExtractStage)

功能	说明
固定间隔策略	按 keyframe_interval_s （默认 2 秒）等间隔提取帧
场景变化策略	ffmpeg scene 检测，在画面变化点提取帧
均匀采样限制	帧数超过 keyframe_max_count （默认 500）时均匀采样
时间戳估算	基于文件名或帧索引计算每帧对应的视频时间

跳过条件: 非视频文件时跳过

Stage 3: OCR 扫描 (OCRScanStage)

功能	说明
逐帧 OCR	使用 RapidOCR（CPU 推理）扫描每帧关键帧中的文字
置信度过滤	低于 ocr_confidence_threshold 的结果自动丢弃
最小长度过滤	过滤过短的无意义文本片段

跳过条件: 无关键帧时跳过

Stage 4: 人脸检测 (FaceDetectStage)

功能	说明
YOLO 逐帧检测	检测每帧中的人脸出现情况
时间线分析	合并相邻相同状态的帧为连续事件（出现/缺失）
短暂缺失过滤	低于 face_missing_threshold_ms 的短暂人脸缺失不报告

跳过条件: 无关键帧时跳过

Stage 5: 音频预处理 (AudioPreprocessStage)

功能	说明
格式转换	任意音频格式转换为 16kHz 单声道 WAV（ASR 标准输入格式）
响度归一化	应用 loudnorm 滤镜统一音频响度

跳过条件: 已有 wav_path 时跳过（视频预处理已生成）

Stage 6: ASR 语音识别 (ASRTranscribeStage)

功能	说明
Paraformer 模式	VAD + ASR + 标点恢复 + 说话人分离四合一，适合多人对话场景
SenseVoice 模式	抗噪增强 ASR + Campplus 滑动窗口声纹聚类说话人分离，适合嘈杂环境
GPU 独占保护	asyncio.Lock 确保同一时刻仅一个 ASR 任务占用 GPU
长段自动拆分	SenseVoice 模式下超长段落基于标点和时长智能拆分
噪声过滤	过滤纯语气词、英文幻觉短语等无效识别结果
句子时间戳	输出 sentence_timestamp，支持后续句子级细粒度对齐

Stage 7: 说话人平滑 (SpeakerSmoothStage)

功能	说明
标签平滑	合并短暂的说话人切换，消除识别抖动
段落预合并	间隔小于 pre_merge_gap_ms 的相同说话人段落合并为一段

Stage 8: 文本纠正 (TextCorrectionStage)

功能	说明
置信度跳过	ASR 置信度达标的段落直接跳过纠正，节省 LLM 调用
四阶段纠正	调用 CorrectorService 执行完整纠正流程（详见 3.3 节）
进度回调	纠正进度实时透传到前端显示

Stage 9: 转写结果构建 (TranscriptBuildStage)

功能	说明
句子细粒度拆分	基于 sub_sentences 按标点拆分为独立句子
按比例分配时间戳	句子时间按字符比例从父段落时间中分配
噪声过滤	纠正后为空字符串的段落自动移除

功能	说明
TranscriptEntry 构建	输出最终的转写条目列表（含原文、纠正文、时间戳、说话人）

3.3 四阶段文本纠正（CorrectorService）

阶段 1：规则预处理

- 英文噪声清理（the, um, uh 等）
- 重复词合并（"那个那个" → "那个"）
- 中文数字年份转阿拉伯数字

↓

阶段 2：热词强制替换

- FlashText 多模式匹配，将预设热词映射为正确写法

↓

阶段 3：pycorrector 轻量纠错

- MacBERT 模型检测同音字/形近字错误

↓

阶段 4：LLM 润色

- Ollama 模型执行 JSON-to-JSON 字幕校对
- 严禁合并/拆分句子，id 必须一一对应
- JSON 解析失败时自动降级为正则提取

3.4 纪要模板系统

功能	说明
动态模板加载	TemplateManager 扫描 templates/ 目录下的.md 文件（YAML frontmatter + prompt 正文），支持运行时热重载
模板元数据	每个模板包含 id、name、description，可通过 API 查询列表
内置模板	5 个：通用（universal，默认）、夕会、周例会、公文、简报摘要
上传时选择	UploadPage 在上传前从 API 加载模板列表，供用户选择
评估时切换	SummaryPanel 支持在结果展示后切换模板重新评估
热重载	POST /templates/reload 无需重启服务即可加载新模板

对应端点: GET /api/v1/templates、POST /api/v1/templates/reload

3.5 动态模板 Map-Reduce 摘要评估 (EvaluatorService)

功能	说明
短文本直接评估	文本长度低于 <code>chunk_size</code> 时，单次 LLM 调用按模板生成纪要
长文本 Map-Reduce	Map 阶段分段并发提取要点，Reduce 阶段合并生成最终纪要
模板注入	模板内容注入 Reduce 阶段 <code>prompt</code> ，控制纪要格式和重点
Pipeline 集成	<code>standard_minutes</code> 主入口在转写完成后自动触发评估
独立评估	支持通过 <code>/evaluate/text/async</code> 对任意文本独立评估

评估输出结构:

- **title:** 纪要标题
- **formatted_content:** Markdown 格式纪要正文（内容由模板决定）

对应端点: POST /api/v1/evaluate/text/async

3.6 多源合规审核 (ComplianceService)

功能	说明
规则文件上传	支持 CSV / XLSX 格式的合规规则文件
四步 CoT 推理	理解规则边界 → 提取证据 → 判断违规 → 输出置信度
规则分组审核	按来源分组: <code>transcript / ocr / vision / mixed</code>
OCR 时间对齐	<code>margin_ms</code> 边界匹配，将 OCR 文本与转写时间线对齐
Map-Reduce 并发	长转写文本分段并发审核，最终合并生成摘要和评分
后处理过滤链	置信度过滤 + 跨规则去重 + 文本验证
违规状态管理	每条违规支持 <code>pending / confirmed / rejected</code> 三种状态
批量状态更新	PATCH 端点支持批量更新违规审核状态
防抖持久化	前端 500ms 防抖聚合状态变更后批量提交后端

违规记录结构:

- **source:** 来源 (`transcript / ocr / vision`)
- **severity:** 严重程度 (`high / medium / low`)
- **confidence:** AI 置信度
- **reasoning:** CoT 推理链（完整判定逻辑）

- **evidence_url / evidence_text:** 证据链接和文本
- **rule_ref:** 对应规则引用
- **status:** 审核状态

对应端点: POST /api/v1/tasks/compliance_audit、PATCH /api/v1/tasks/{task_id}/compliance/violations

3.7 文件持久化 (PersistenceService)

功能	说明
原子写入 SHA-256 哈希索引	tempfile + rename 确保写入不会产生损坏文件 维护文件哈希到 task_id 的映射, 支持去重查询; 服务启动时自动清理指向不存在任务的 stale 条目
JSON 持久化	transcript / evaluation / compliance / ocr_results / visual_events
多媒体持久化 启动恢复	audio / video / frames / synthesis 分别存储 scan_completed_tasks 扫描磁盘, 恢复内存任务列表

持久化目录结构:

```

uploads/{task_id}/
├── meta.json           # 任务元数据
├── transcript.json      # 转写结果
├── evaluation.json     # 评估结果
├── compliance.json     # 合规审核结果
├── ocr_results.json    # OCR 扫描结果
├── visual_events.json  # 视觉事件
├── synthesis.mp3       # TTS 合成音频 (按需生成)
├── synthesis_result.json # 合成耗时元数据 (按需生成)
├── audio.{ext}         # 提取的音频
├── video.{ext}         # 原始视频 (视频任务)
└── frames/            # 关键帧图片
  
```

3.8 任务管理 (TaskStore)

功能	说明
内存任务存储	轻量级内存 store, 支持 LRU 淘汰 (task_max_in_memory)

功能	说明
四种任务类型	标准纪要任务、纯转写任务、纯文本评估任务、合规审核任务
超时保护	<code>asyncio.wait_for</code> 包装，超过 <code>task_timeout_seconds</code> 自动中止
进度计算	按处理阶段分段计算百分比，实时上报前端
重新转写	支持基于已有媒体文件重新执行转写（清除下游评估和合规结果）

任务状态流转:

PENDING → PROCESSING_ASR → EXTRACTING_FRAMES → SCANNING_VISUAL
 → CORRECTING → EVALUATING → AUDITING → COMPLETED
 ↘ FAILED

3.9 TTS 音频重塑 (SynthesisService)

功能	说明
多说话人音色	不同说话人自动分配不同 ChatTTS 音色（可通过 <code>voice_map</code> 覆盖）
说话人段落合并	连续相同说话人的段落合并后一次性合成，语调更自然
LLM 口语改写	合成前用 LLM 将转写稿改写为自然口语（可配置开关和语气能量级别）
批次 VRAM 刷新	按批次合成，每批完成后清空 VRAM 缓存，防止长转写 OOM
模型独占互斥	通过 <code>ModelManager</code> 卸载 ASR，ChatTTS 独占显存，合成后驻留等待
格式输出	多段 WAV 合并为单一 MP3 文件 (<code>synthesis.mp3</code>)
异步执行	<code>synthesize</code> 返回 202 后在后台合成，通过 <code>status</code> 端点轮询进度；重复触发返回 409，LLM 任务运行中返回 503

对应端点: POST `/api/v1/tasks/{task_id}/synthesize`、GET `/api/v1/tasks/{task_id}/synthesis/status`、GET `/api/v1/tasks/{task_id}/synthesis`

3.10 前端工作区

三栏布局

Navbar (品牌 + 主题切换)			
左栏 420px	中栏 (自适应)	右栏 380px (条件)	
MediaPlayer (音视频)	Tab: 转写结果 TranscriptToolbar TranscriptList (虚拟滚动)	EvidenceDetailPanel (违规证据详情)	
SummaryPanel (智能摘要)			
[可折叠]	Tab: 违规报告 ViolationList (过滤/搜索/批量)		
Compliance Panel [可折叠]			
Synthesis Panel [可折叠]			

音视频播放器

功能	说明
双模式切换	根据 <code>mediaType</code> 自动切换 <code>video</code> 元素或 <code>audio + 波形图</code>
波形可视化	<code>WaveSurfer.js</code> 渲染音频波形
播放控制	播放/暂停、进度拖拽、倍速调节、音量控制
循环播放	支持设置循环区间，审核违规时自动设置上下文区间
播放同步	<code>requestAnimationFrame</code> 高精度同步播放时间到 <code>Store</code>

转写结果展示

功能	说明
虚拟滚动	<code>React Virtuoso</code> 高性能渲染，支持数千条转写记录
聊天气泡布局	按说话人左右交替排列，当前播放块高亮
句子级时间对齐	每个句子可独立点击跳转到对应音频位置
自动滚动	播放时自动滚动到当前位置，二分查找定位
原文/修正文切换	一键切换显示 <code>ASR</code> 原始文本或纠正后文本
文本编辑	支持在线编辑纠正后的文本
全文搜索	实时搜索匹配并高亮显示
说话人筛选	按说话人显隐过滤，专注特定发言者
说话人重命名	批量将 <code>spk_0</code> 等标签重命名为具体姓名
重新转写	支持修改热词后重新执行 <code>ASR + 纠正</code>

功能	说明
多格式导出	支持导出为 SRT 字幕 / Word 文档 / PDF 文档

智能摘要面板

功能	说明
自动触发	<code>standard_minutes</code> 完成后服务端已生成纪要，直接展示；轮询完成时优先读取服务端结果 下拉菜单选择纪要模板（模板 > 1 条时显示），切换后点“重新评估”生效 显示 <code>title</code> （标题）和 <code>formatted_content</code> （Markdown 正文， <code>pre-wrap</code> 渲染） 使用当前选中模板重新调用 <code>/evaluate/text/async</code> ，不依赖 <code>rerun-evaluation</code> 端点
模板选择	
纪要展示	
重新评估	

合规审核面板

功能	说明
规则文件上传	支持 CSV / XLSX 规则文件
合规分数展示	环形进度条 + 违规分级统计（高/中/低）
审核摘要	AI 生成的合规审核总结
重新审核	支持修改规则后重新执行合规审核

音频重塑面板

功能	说明
合成触发	点击“合成对话音频”按钮，调用 <code>synthesize</code> 接口（202 异步），2 秒间隔轮询合成状态
持久化检测	页面挂载时通过 <code>getTaskResults</code> 检测 <code>has_synthesis</code> ，已合成则自动展开面板
内嵌播放器	自定义播放控件（播放/暂停按钮 + 进度条 + 时间显示），无需跳转
下载	一键下载合成 MP3
状态徽章	已合成时面板标题显示“已合成”绿色徽章
重新合成	合成完成后按钮切换为“重新合成”

违规审核工作台

功能	说明
统计仪表板	高风险 / 疑似 / 待审 / 合规度四项核心指标

功能	说明
三级过滤	严重程度（高/中/低）+ 来源（语音/OCR/视觉）+ 状态（待审/已确认/已忽略）
搜索	匹配违规原因、原文、规则内容、证据文本
违规卡片	显示时间戳 / 来源 / 严重程度 / 说话人 / 状态徽章
AI 推理展示	可折叠查看 CoT 完整判定逻辑
证据展示	语音原文引用 / OCR 文本块 + 截图 / 视觉截图 + 描述
规则引用	带 tooltip 展示完整规则内容
状态操作	确认违规 / 误报忽略 / 重新审核（重置为待审）
批量操作	B 键开关批量模式，Ctrl+A 全选，批量确认/忽略
证据详情面板	右侧 380px 抽屉，展示完整证据信息 + 截图缩放 + 转录上下文
时间跳转	点击违规时间戳自动跳转播放器并设置循环区间

键盘快捷键（违规审核）

按键	功能
Space	播放/暂停
Enter	确认当前违规
Delete / Backspace	忽略当前违规
上/下箭头	导航上/下一条违规
B	切换批量操作模式
Ctrl+A	批量模式下全选
Esc	关闭证据详情面板或退出批量模式

3.11 LLM 客户端（OllamaClient）

功能	说明
流式响应	stream=True 逐 token 接收，避免长推理超时
全局并发控制	Semaphore 限制最大并发 LLM 请求数（默认 3）
重试机制	指数退避重试（ $2^{\text{attempt}} * \text{retry_delay}$ ），默认 2 次
动态超时	大文本 prompt 可覆盖默认 read_timeout
思考模式	think 参数控制：None（默认）/ False（批量纠正）/ True（深度推理）
动态 num_ctx	纠正 4096 / 评估 8192 / 通用 32768，按场景适配显存

功能	说明
口语改写专属客户端	TTS 改写使用独立 OllamaClient 实例，与主 LLM 流程隔离

3.12 GPU 模型管理器 (ModelManager)

功能	说明
插件式注册	register_loader 注册各模型的加载/卸载函数及显存估算
互斥独占	acquire(model_type) 作为异步上下文管理器，自动卸载其他模型再加载目标模型
显存估算	记录各模型估算显存占用，提供预算视图
主动释放	unload / unload_all 执行卸载 + CUDA cache 清理
当前管理范围	ASR 模型 + TTS (ChatTTS) 模型，二者不可同时驻留

3.13 健康检查

功能	说明
整体状态	顶层 status: healthy / degraded / unhealthy (unhealthy 时接口返回 503)
组件状态	asr / llm / tts 三个组件对象，各含 status (ok / degraded / down) 与 detail
任务统计	tasks: active / completed / failed / synthesis_running
VRAM 水位	返回 ModelManager 管理的模型列表、估算显存占用及预算上限
前端状态页	前端 /health 页面可视化展示以上信息，10 秒自动刷新，入口位于首页与工作区 Navbar

对应端点: GET /api/v1/health

四、API 端点汇总

方法	路径	功能
GET	/api/v1/uploads/{hash}	查询或创建分片上传会话（断点续传预检）
PATCH	/api/v1/uploads/{hash}	上传数据块（末块自动触发 <code>standard_minutes</code> 任务）
POST	/api/v1/tasks/standard_minutes	主入口：上传文件创建完整纪要任务
POST	/api/v1/tasks/transcript	轻量上传：仅 ASR + 纠正，不含摘要
GET	/api/v1/tasks/lookup	按 SHA-256 查询已有任务
GET	/api/v1/tasks/{task_id}	查询任务状态和进度
GET	/api/v1/tasks/{task_id}/results	获取任务完整结果（转写 + 评估 + 合规 + <code>is_synthesis</code> ）
GET	/api/v1/tasks/{task_id}/media	获取原始媒体文件（视频优先，回退音频）
GET	/api/v1/tasks/{task_id}/frames/{frame_id}	获取关键帧图片
DELETE	/api/v1/tasks/{task_id}	作废任务缓存（不删除磁盘文件）
POST	/api/v1/tasks/{task_id}/rerun-transcript	重新转写
POST	/api/v1/tasks/{task_id}/synthesize	触发 TTS 多说话人音频合成
GET	/api/v1/tasks/{task_id}/synthesize/status	查询合成任务状态
GET	/api/v1/tasks/{task_id}/synthesize/audio	下载已合成的 MP3 音频
GET	/api/v1/templates	获取所有可用纪要模板列表
POST	/api/v1/templates/reload	热重载纪要模板目录
POST	/api/v1/evaluate/text/async	提交纯文本评估任务（支持 <code>template_id</code> ）
POST	/api/v1/evaluate/transcript/async	接收第三方转写结果并评估
POST	/api/v1/tasks/compliance_audit	提交合规审核任务
PATCH	/api/v1/tasks/{task_id}/compliance_audit/batch	批量更新合规审核状态
GET	/api/v1/health	服务健康检查（组件状态 + 任务统计 + VRAM 水位，unhealthy 时 503）

五、并发与容错机制

机制	说明
GPU 独占锁 ModelManager 互斥	ASR 推理通过 <code>asyncio.Lock</code> 保证串行执行 ASR 与 TTS 通过 <code>acquire()</code> 机制保证不同时驻留
LLM 并发信号量	全局 <code>Semaphore</code> 限制并行 LLM 请求，防止 Ollama 过载

机制	说明
指数退避重试	LLM 调用失败后按 2^n 间隔重试
JSON 解析降级	LLM 输出非法 JSON 时自动降级为正则提取
任务超时保护	<code>asyncio.wait_for</code> 包装，超时自动标记失败
流式响应	避免长推理时 HTTP 连接超时
原子文件写入	<code>tempfile + rename</code> 防止持久化中断导致数据损坏
前端竞态保护	轮询完成时先加载服务端 <code>evaluation</code> 写入 <code>store</code> ，再设置 <code>rawEntries</code> ，防止 <code>SummaryPanel</code> 用默认模板重复触发
防抖批量持久化	违规状态变更 500ms 聚合后批量提交
分片上传重试	每块最多重试 3 次，重试前向服务端查询当前 <code>offset</code> ，避免重复上传
TTS OOM 保护	同时捕获 <code>CUDA OutOfMemoryError</code> 与 <code>Python MemoryError</code> ，GPU 和 CPU 两种推理模式均有防护

六、显存优化策略

策略	说明
FP16 推理	ASR 使用 <code>float16</code> 推理，减少约 50% 显存占用
ModelManager 互斥	ASR 与 TTS 不同时驻留，共用显存空间
动态 <code>num_ctx</code>	按场景分配上下文窗口：纠正 4096 / 评估 8192 / 审核 32768
Map-Reduce	长文本分段处理，避免单次 <code>prompt</code> 撑满上下文
<code>num_predict</code> 限制	限制 LLM 输出 <code>token</code> 数，避免无限生成
TTS 批次刷新	每批 TTS 合成后清空 VRAM 缓存，防长转写 OOM
OCR CPU 推理	RapidOCR 使用 CPU，不占用 GPU 显存
批量大小控制	ASR <code>batch_size_s</code> 上限 60s，防止 GPU OOM